



浙江工业大学
ZHEJIANG UNIVERSITY OF TECHNOLOGY

知识图谱课程报告

ZJUT 伴学 Bot：基于 OneKE 和 Neo4j 的智能问答系统

学院 信息工程学院

专业 智能科学与技术

学号 302023511143

姓名 张皓博

2026 年 5 月 28 日

摘 要

本实验构建了“ZJUT 伴学 Bot”——一个面向浙江工业大学信息学院培养计划的智能问答机器人，涵盖知识抽取、图谱构建、检索增强生成（RAG）和学生长期记忆四个核心阶段。系统聚焦自动化、智能科学与技术、通信工程三个专业，并扩展浙江工业大学基础信息、学习资源和学科竞赛知识，包含 289 个实体节点（3 个专业、139 门课程、15 个能力、12 个职业方向、2 项学科竞赛等）和 561 条知识关系。首先，利用 OneKE 开源工具从培养计划 PDF 文档中自动抽取结构化知识三元组（如“自动化专业-开设-自动控制原理”），并人工补充校园与竞赛相关三元组，通过 Schema 设计和质量过滤获得高质量知识。其次，采用 Neo4j 图数据库存储知识图谱，支持高效的 Cypher 查询和图谱可视化。最后，实现基于 RAG 的 LLM 问答系统，并将长期记忆从本地 JSON 画像升级为 Graphiti+Neo4j 图记忆：当前 Web 主问答模型配置为 DeepSeek deepseek-v4-flash，Graphiti 内部结构化抽取模型采用 DashScope 兼容模式下的 qwen-plus，嵌入模型采用 DashScope text-embedding-v3（1024 维）。系统提供业务化 Web 工作台和命令行双入口，Web 端保留问答、知识图谱查看、对话清空和长期记忆清空能力，成绩单等个人材料通过本地脚本或对话显式输入维护，避免在公开 Web 界面暴露上传入口。系统能够回答专业课程、学分要求、前置课程、能力培养、竞赛规划等问题，并根据学生长期记忆生成个性化学业规划建议。

目录

1	引言	1
1.1	研究背景	1
1.2	研究目标	1
1.3	技术路线	1
2	知识抽取	2
2.1	数据源分析	2
2.2	Schema 设计	2
2.3	OneKE 知识抽取	2
2.4	质量保证与后处理	3
3	知识图谱构建	4
3.1	图谱建模	4
3.2	数据导入	4
3.3	索引优化	5
3.4	图谱统计	5
4	RAG 问答系统	6
4.1	系统架构	6
4.2	向量检索模块	7
4.2.1	三元组向量化	7
4.2.2	相似度检索	7
4.3	Neo4j 图谱查询	8
4.4	混合检索策略	8
4.5	答案生成	9
4.6	学生长期记忆模块	9
4.7	Graphiti 图记忆系统升级	12
4.7.1	Graphiti 内部模型与关键配置	12
4.7.2	记忆图的数据主体	13
4.7.3	写入与检索流程	14
4.7.4	本地部署与 Neo4j 可视化	15

5	系统展示	16
5.1	系统架构总览	16
5.2	知识图谱可视化	16
5.3	Web 交互界面	16
6	总结与展望	18
6.1	研究总结	18
6.2	系统特色	20
6.3	改进方向	20

1 引言

1.1 研究背景

知识图谱作为一种结构化的语义知识库，通过实体、关系和属性的三元组形式表示现实世界的知识^[1]。近年来，知识图谱在智能问答、推荐系统、语义搜索等领域展现出巨大应用价值。在教育信息化领域，构建面向培养计划、课程体系的知识图谱，能够为学生提供智能化的学业规划和课程咨询服务。

传统的知识图谱构建依赖人工标注，效率低且难以扩展。随着大语言模型的发展，基于 LLM 的知识抽取工具（如 OneKE^[2]）能够自动从非结构化文本中提取结构化知识，大幅降低构建成本。同时，检索增强生成（RAG）技术^[3]通过结合知识检索与生成模型，显著提升了问答系统的准确性和可解释性。

1.2 研究目标

本实验旨在构建一个完整的教育领域知识图谱问答系统，具体目标包括：

- 使用 OneKE 工具自动从浙江工业大学信息学院培养计划文档中抽取知识三元组
- 采用 Neo4j 图数据库^[4]存储和管理知识图谱
- 实现基于向量检索的 RAG 问答系统，支持自然语言交互
- 引入学生长期记忆机制，使系统能够根据学生画像提供个性化学习建议
- 验证系统在专业课程查询、学分要求、能力培养等场景下的有效性

1.3 技术路线

本实验采用三阶段技术路线：

1. 知识抽取阶段：PDF 文档解析 → Schema 设计 → OneKE 抽取 → 质量过滤
2. 图谱构建阶段：实体建模 → Neo4j 导入 → 索引优化 → 图谱可视化
3. RAG 问答阶段：向量化 → 混合检索 → Prompt 构建 → LLM 生成

完整技术流程为：培养计划 PDF 文档 → OneKE 自动抽取 → Neo4j 图谱存储 → 向量检索 + 图谱查询 → 学生记忆注入 → LLM 生成个性化答案。

2 知识抽取

2.1 数据源分析

实验数据主要来源于浙江工业大学信息学院培养计划 PDF 文档，并补充浙江工业大学基础信息、校园学习资源和学科竞赛资料，涵盖自动化、智能科学与技术、通信工程三个专业的详细信息，包括：

- 专业基本信息：所属学院、学分要求、培养目标
- 课程体系：课程名称、学分、开设学期、课程类型
- 课程关系：前置课程、后续课程
- 能力培养：专业能力、职业方向
- 校园知识：学校、校区、图书馆、教务系统、第二课堂等信息
- 竞赛知识：全国大学生智能汽车竞赛、全国大学生电子设计竞赛及其支撑课程

2.2 Schema 设计

Schema 定义了知识图谱的本体结构，包括实体类型和关系类型。本实验设计的 Schema 如表 1 所示。

类型	具体内容
实体类型	专业、课程、学院、能力、方向、学分、学校、校区、学科竞赛
关系类型	开设、属于、要求、培养、适用于、前置课程、支撑、适合参与

表 1 知识图谱 Schema 设计

2.3 OneKE 知识抽取

OneKE 是一个基于大语言模型的 Schema 引导式知识抽取系统^[2]，支持从非结构化文本中自动识别实体和关系。

如图 1 所示，OneKE 采用 Docker 化部署方式，通过 LLM Agent 实现 Schema 引导的知识抽取。系统核心包括：（1）Schema 定义模块，用户可自定义实体类型和关系类型；（2）LLM 推理引擎，基于大语言模型进行实体识别和关系抽取；（3）后处理模块，对抽取结果进行置信度评分和格式化输出。相比传统基于规则或监督学习的方法，OneKE 具有零样本学习能力，无需大量标注数据即可适应新领域。

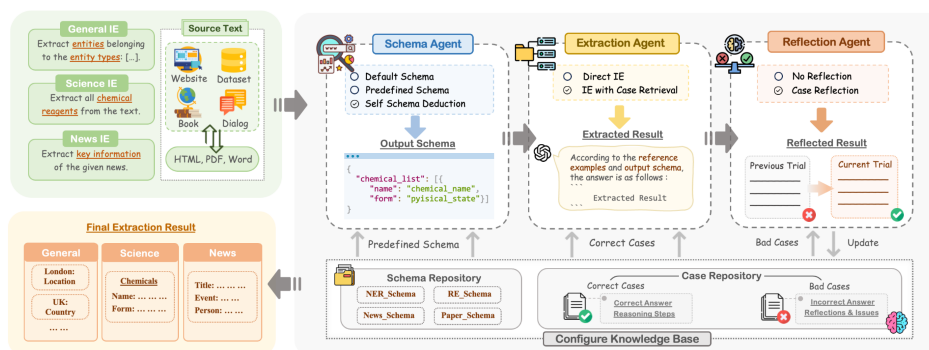


图 1 OneKE 系统架构图

本实验的抽取流程如下：

“OneKE 抽取流程”

```

1 from oneke import OneKEExtractor
2 import pdfplumber
3
4 # 1. PDF文本提取
5 with pdfplumber.open("培养计划.pdf") as pdf:
6     text = "".join([page.extract_text() for page in pdf.pages])
7
8 # 2. Schema定义
9 schema = {
10     "entities": ["专业", "课程", "学院", "能力", "方向"],
11     "relations": ["开设", "属于", "要求", "培养", "前置课程"]
12 }
13
14 # 3. OneKE抽取
15 extractor = OneKEExtractor(model="oneke-base")
16 raw_triples = extractor.extract(text, schema, language="zh")
17
18 # 4. 置信度过滤
19 triples = [t for t in raw_triples if t['confidence'] > 0.75]

```

2.4 质量保证与后处理

为保证抽取质量，采取以下措施：

- 置信度过滤：设置阈值 0.75，过滤低置信度三元组
- 实体归一化：统一实体表述（如“自动化”→“自动化专业”）
- 关系标准化：映射同义关系（如“包含”→“开设”）

- **人工抽查：**随机抽取 100 条样本进行人工验证

实验当前共整理 561 条高质量三元组，涵盖专业、课程、能力、职业方向、校园资源和学科竞赛等多个维度的知识。

3 知识图谱构建

3.1 图谱建模

Neo4j 是一个原生图数据库，采用属性图模型存储数据^[4]。本实验设计的图谱模型包括以下节点和关系：**节点类型：**

- Major（专业）：属性包括 name、credits
- Course（课程）：属性包括 name、credits、semester
- College（学院）：属性包括 name
- Skill（能力）：属性包括 name
- Direction（方向）：属性包括 name

关系类型：

- BELONGS_TO：专业属于学院
- OFFERS：专业开设课程
- REQUIRES：专业要求学分
- CULTIVATES：专业培养能力
- PREREQUISITE：课程前置关系

3.2 数据导入

使用 Neo4j Python 驱动批量导入三元组数据：

”Neo4j 批量导入”

```
1 from neo4j import GraphDatabase
2
3 class Neo4jLoader:
4     def __init__(self, uri, user, password):
5         self.driver = GraphDatabase.driver(uri, auth=(user, password))
6
7     def load_triples(self, triples):
```



```
8         with self.driver.session() as session:
9             # 创建节点
10            session.run("""
11                UNWIND $triples AS t
12                MERGE (s {name: t.subject, type: t.subject_type})
13                MERGE (o {name: t.object, type: t.object_type})
14            """, triples=triples)
15
16            # 创建关系
17            session.run("""
18                UNWIND $triples AS t
19                MATCH (s {name: t.subject})
20                MATCH (o {name: t.object})
21                MERGE (s)-[r:RELATION {type: t.relation}]->(o)
22            """, triples=triples)
```

3.3 索引优化

为提升查询性能，创建全文索引：

”创建索引”

```
1 session.run("""
2     CREATE FULLTEXT INDEX entity_index
3     FOR (n) ON EACH [n.name]
4 """)
```

3.4 图谱统计

构建完成的知识图谱包含：

- 节点总数：289 个（3 个专业、139 门课程、1 个学院、15 个能力、12 个职业方向、15 个课程类别、2 项学科竞赛等）
- 关系总数：561 条
- 关系类型：32 种（开设、属于、要求、培养、适用于、前置课程、支撑、适合参与等）
- 平均度数：3.9（每个节点平均连接约 3.9 条边）

4 RAG 问答系统

4.1 系统架构

RAG (Retrieval-Augmented Generation) 通过检索相关知识增强生成模型^[3]。本系统采用混合检索策略，结合向量检索和图谱查询。系统工作流程为：用户提问 → 向量检索 (BGE+Faiss) 获取语义相似三元组 → 图谱查询 (Neo4j+Cypher) 获取结构化知识 → 混合排序 Top-K 结果 → 构建 Prompt → LLM 生成答案。

在 OneKE 知识抽取 Pipeline 的基础上，本项目进一步形成了从培养计划 PDF 到课程知识图谱、RAG 问答推理、Graphiti 长期记忆和 Web/CLI 应用的端到端系统流水线，如图 2 所示。该图同时展示了模型平面和存储平面：Web 主问答模型当前采用 DeepSeek deepseek-v4-flash，Graphiti 内部使用 qwen-plus 进行对话 episode 结构化抽取，并调用 DashScope text-embedding-v3 生成 1024 维嵌入；课程图谱、Neo4j 记忆图和本地 JSON 记忆共同构成系统的可复现存储基础。

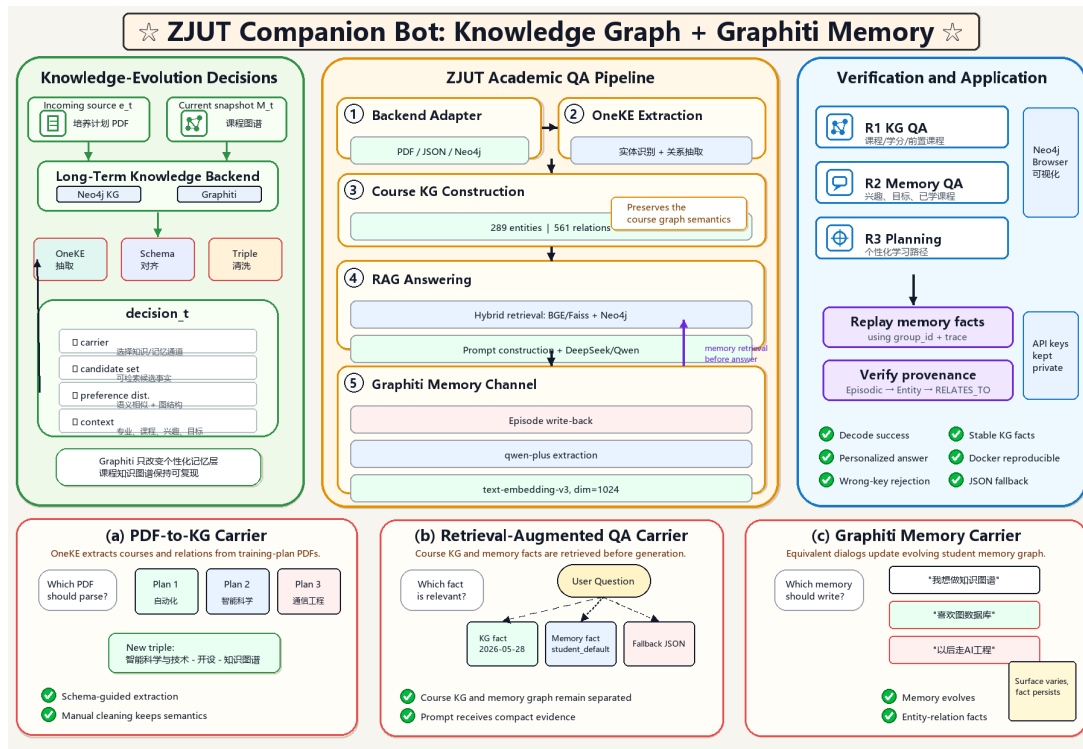


图 2 ZJUT 伴学 Bot 整体 Pipeline 架构图 (MemMark 风格)

4.2 向量检索模块

4.2.1 三元组向量化

使用 BGE-small-zh-v1.5 模型^[5]对三元组进行向量化。BGE 是智源研究院开源的中文语义向量模型，在多个中文语义匹配任务上达到 SOTA 性能。

”向量化与索引构建”

```

1 from sentence_transformers import SentenceTransformer
2 import faiss
3
4 class VectorRetriever:
5     def __init__(self, model_name='BAAI/bge-small-zh-v1.5'):
6         self.model = SentenceTransformer(model_name)
7         self.index = None
8         self.triples = []
9
10    def build_index(self, triples):
11        self.triples = triples
12        # 三元组文本化
13        texts = [f"{t['subject']} {t['relation']} {t['object']}"
14                 for t in triples]
15
16        # 向量化
17        embeddings = self.model.encode(texts, normalize_embeddings=True)
18
19        # 构建Faiss索引
20        dimension = embeddings.shape[1]
21        self.index = faiss.IndexFlatIP(dimension)
22        self.index.add(embeddings.astype('float32'))

```

4.2.2 相似度检索

Faiss^[6]是 Facebook 开源的高效向量检索库，支持十亿级向量的毫秒级检索。本实验使用内积（Inner Product）相似度：

$$\text{similarity}(q, d) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (1)$$

其中 $\mathbf{q}$ 为问题向量， $\mathbf{d}$ 为三元组向量。

4.3 Neo4j 图谱查询

除向量检索外，系统还通过 Cypher 查询获取实体的结构化信息：

”图谱查询”

```

1 class Neo4jRetriever:
2     def retrieve_by_entity(self, entity_name, hop=1):
3         with self.driver.session() as session:
4             result = session.run(f"""
5                 MATCH path = (n {{name: $name}}) -[*1..{hop}]- (m)
6                 RETURN
7                     startNode(relationships(path)[0]).name as subject ,
8                     type(relationships(path)[0]) as relation ,
9                     endNode(relationships(path)[0]).name as object
10                LIMIT 20
11                """, name=entity_name)
12     return [dict(record) for record in result]
```

4.4 混合检索策略

结合向量检索（语义相似）和图谱查询（结构信息）：

”混合检索”

```

1 class HybridRetriever:
2     def retrieve(self, question, top_k=15):
3         # 1. 向量检索 Top-10
4         vector_results = self.vector_ret.retrieve(question, top_k=10)
5
6         # 2. 提取关键实体
7         entities = self._extract_entities(question)
8
9         # 3. Neo4j 图谱查询
10        graph_results = []
11        for entity in entities[:2]:
12            graph_results.extend(
13                self.neo4j_ret.retrieve_by_entity(entity, hop=1)
14            )
15
16        # 4. 去重合并
17        all_triples = self._merge_results(vector_results, graph_results)
18        return all_triples[:top_k]
```

4.5 答案生成

系统通过统一的 LLMClient 封装不同大语言模型接口，可接入 DeepSeek、Qwen、智谱或 OpenAI 兼容模型生成答案。当前 Web 聊天配置使用 deepseek-v4-flash，同时为 OpenAI 兼容客户端设置请求超时与最大输出长度，避免外部模型服务异常导致 Web 请求长时间悬挂。Prompt 设计如下：

”Prompt 构建与生成”

```
1 class RAGChatBot:
2     def ask(self, question):
3         # 1. 检索相关知识
4         retrieved = self.retriever.retrieve(question, top_k=15)
5
6         # 2. 格式化上下文
7         context = "\n".join([
8             f"{i}. {t['subject']} {t['relation']} {t['object']}"
9             for i, t in enumerate(retrieved, 1)
10        ])
11
12        # 3. 构建 Prompt
13        prompt = f"""用户问题: {question}
14
15        知识图谱信息:
16        {context}
17
18        请基于上述知识回答问题。"""
19
20        # 4. LLM生成
21        messages = [
22            {"role": "system", "content": "你是浙工大信息学院智能助手"},
23            {"role": "user", "content": prompt}
24        ]
25        answer = self.llm.chat(messages, temperature=0.3)
26        return answer
```

4.6 学生长期记忆模块

传统 RAG 问答系统主要面向单轮或短上下文对话，能够回答“课程有哪些”“学分是多少”等事实性问题，但难以持续理解学生的个人背景、兴趣方向和学习进度。为使 ZJUT 伴学 Bot 从一次性问答工具进一步扩展为“同步成长”

的学习智能体，本实验在 RAG 问答链路中加入本地 JSON 长期记忆模块。其核心思想是：在每轮对话前读取学生画像并注入 Prompt，在回答后从用户表达中抽取可长期保存的信息，更新学生画像，从而实现跨轮次、跨会话的个性化问答。

长期记忆模块主要存储以下信息：

- **基础画像**：学生专业、年级等静态背景信息
- **兴趣方向**：如人工智能、机器学习、知识图谱、嵌入式、通信等方向偏好
- **学习状态**：已学课程、薄弱课程、关注课程
- **发展目标**：如考研、就业、竞赛、科研训练等目标
- **近期问题**：保留最近若干轮问题，辅助生成阶段理解学生连续需求

系统的数据流如下：

Question+Memory+KG Context → Prompt → LLM Answer → Response ; Memory Update_{async}

在当前实现中，回答生成链路与 Graphiti 写入链路被解耦：系统会在回答前检索长期记忆并注入 Prompt，但 LLM 生成答案后先返回给 Web 前端，再通过后台线程将本轮“问题—回答”写入 Graphiti。这样可以避免 Graphiti 实体抽取、向量嵌入和 Neo4j 写入过程阻塞用户界面；代价是下一轮对话未必立刻读到刚刚写入的最新 episode。

本实验采用本地 JSON 文件保存学生画像，避免引入额外数据库，便于课程实验部署和结果复现。记忆文件结构如下：

”学生长期记忆 JSON 结构”

```
1 {  
2   "student_id": "default",  
3   "major": "智能科学与技术专业",  
4   "grade": "大一",  
5   "interests": ["机器学习", "深度学习", "知识图谱"],  
6   "goals": ["考研", "竞赛"],  
7   "learned_courses": ["机器学习", "知识图谱"],  
8   "weak_courses": ["概率论"],  
9   "recent_questions": [],  
10  "summaries": [],  
11  "updated_at": "2026-05-09 13:53:08"  
12 }
```

在实现上，系统将长期记忆拆分为两个核心组件：**MemoryStore** 负责 JSON 文件的读取、合并、保存和清空；**MemoryExtractor** 负责从用户自然语言中抽取专业、年级、兴趣、目标、已学课程和薄弱课程等信息。其核心流程如下：

”长期记忆注入与更新流程”

```
1 class LLMChatBot:
2     def ask(self, question):
3         # 1. 检索知识图谱相关三元组
4         triples = self.retriever.retrieve_relevant_knowledge(question)
5         knowledge_context = self.retriever.format_knowledge_context(triples)
6
7         # 2. 读取学生长期记忆
8         memory_context = self.memory_store.format_for_prompt()
9
10        # 3. 构建包含学生画像的 Prompt
11        prompt = f"""
12        用户问题: {question}
13        学生长期记忆: {memory_context}
14        知识图谱信息: {knowledge_context}
15        请结合学生画像和知识图谱回答。
16        """
17
18        # 4. 调用LLM生成个性化答案
19        answer = self.llm_client.chat(prompt)
20
21        # 5. 从本轮对话抽取新记忆并持久化
22        memory = self.memory_extractor.extract(question, answer)
23        self.memory_store.apply_extraction(question, memory)
24        return answer
```

例如，当学生表达“我是智科专业大一学生，已经学过机器学习和知识图谱，想参加智能车竞赛”时，系统会自动更新其专业、年级、已学课程、兴趣方向和竞赛目标。之后学生再询问“我还应该补哪些课”，系统不再只给出通用课程列表，而是结合其已有基础和目标竞赛，优先推荐自动控制、单片机、嵌入式系统、图像处理等支撑课程。由此，系统具备了“记住学生状态—结合图谱推理—动态生成建议”的闭环能力。

4.7 Graphiti 图记忆系统升级

在本地 JSON 记忆系统的基础上，本项目进一步将长期记忆升级为基于 Graphiti 和 Neo4j 的图记忆系统。JSON 记忆适合保存固定字段的学生画像，但其表达能力受限：它难以自然表示“学生—关注方向—课程—项目—回答偏好”之间的多跳关系，也难以支持基于语义相似度的动态检索。Graphiti 的优势在于将对话过程视为不断到来的 **episode**，并从 **episode** 中抽取实体、关系和事实边，形成可持续演化的时序知识图谱。

图 3 给出了升级后的端到端架构。用户问题首先进入 LLM 问答链路，系统同时检索课程知识图谱上下文和 Graphiti 长期记忆上下文；LLM 生成回答后，本轮“问题—回答”会作为新的 **episode** 异步写回 Graphiti，Graphiti 再调用内部抽取模型和嵌入模型，将对话转化为实体、事实边和向量索引。

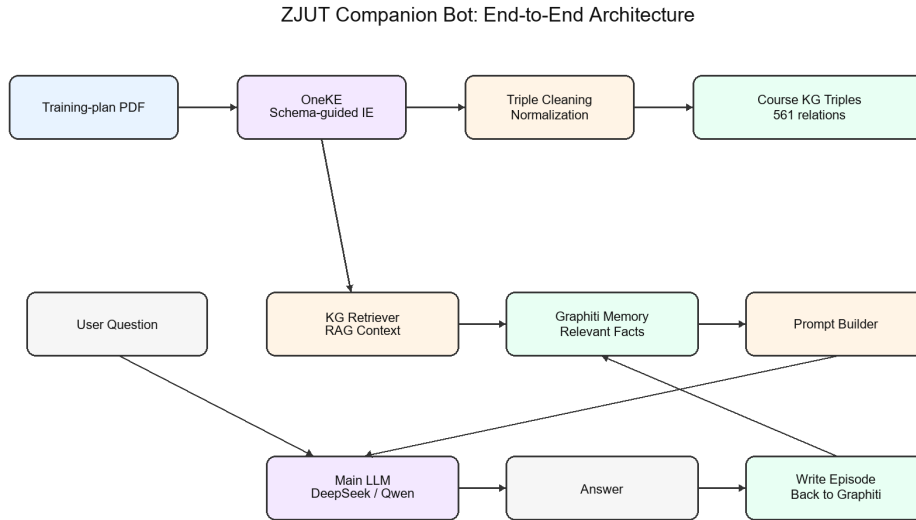


图 3 引入 Graphiti 后的 ZJUT 伴学 Bot 端到端系统架构

4.7.1 Graphiti 内部模型与关键配置

本实验中的 Graphiti 记忆模块由 GraphitiMemoryStore 进行封装。由于 Graphiti 本身是异步 Python 库，而 Flask 和命令行问答流程是同步调用方式，因此系统在适配层中维护事件循环与线程锁，将 `add_episode()` 和 `search()` 包装为可在同步代码中调用的接口。Web 问答路径进一步把 `add_episode()` 放入后台线程执行，从而保证用户请求主要等待主问答 LLM 而不是等待记忆抽取完成。其关键配置如表 2 所示。

模块	实际配置	作用说明
Graphiti 版本	graphiti-core==0.20.4	固定版本以保证与本地 Neo4j 5.20 环境可复现；若使用 Neo4j 5.26+, 可进一步升级 Graphiti 版本。
图数据库	Neo4j 5.20/5.26	存储 Episodic 对话节点、Entity 实体节点和 RELATES_TO 事实关系。
Graphiti 内部 LLM	qwen-plus	通过 DashScope OpenAI-compatible 接口调用, 用于从对话 episode 中抽取实体、关系和事实。
主问答 LLM	deepseek-v4-flash	用于根据“用户问题+课程知识图谱+长期记忆”生成最终自然语言回答; 通过 DeepSeek OpenAI-compatible 接口调用。
嵌入模型	text-embedding-v3	DashScope 文本向量模型, 输出维度为 1024, 用于事实边和实体名称的语义检索。
记忆命名空间	student_default	对应 Graphiti 的 group_id, 用于区分不同学生或不同实验会话的长期记忆。
检索数量	Top-8	每轮问答前默认检索最多 8 条相关长期记忆事实注入 Prompt。
降级机制	Local JSON Memory	当 Neo4j、Graphiti 或外部 API 不可用时, 系统自动回退到本地 JSON 记忆, 保证基本可运行。

表 2 Graphiti 图记忆模块关键配置

这里需要区分两个“模型”：其一是**主问答模型**，负责面向用户生成回答，当前本地配置使用 `deepseek-v4-flash`；其二是**Graphiti 内部抽取模型**，负责把对话文本转化为结构化记忆，当前配置使用 DashScope 兼容模式下的 `qwen-plus`。二者可以相同，也可以不同。本实验选择将二者解耦，是因为 Graphiti 抽取阶段需要较稳定的结构化输出能力，而主问答阶段更关注回答质量、速度和成本。

4.7.2 记忆图的数据主体

Graphiti 写入 Neo4j 后，记忆图的主体包括三类对象：

- **Episodic 节点**：保存原始对话片段，即一轮用户问题和助手回答。系统写入时使用 `source=EpisodeType.message`, `episode_body` 格式为“Student: 问题 + Assistant: 回答”。
- **Entity 节点**：Graphiti 从对话中抽取出的实体，例如“学生”“智能科学与技术专业”“知识图谱”“图数据库”“RDF/属性图”等。
- **RELATES_TO 关系边**：实体之间的事实关系，边上保存 `fact`、`valid_at`、`group_id` 以及 `fact_embedding` 等属性。问答时实际注入 Prompt 的长期记忆主要来自这些事实边。

因此，在 Neo4j Browser 中查看长期记忆时，最常用的查询是：

”Graphiti 长期记忆事实查询”

```
1 MATCH (n:Entity)-[r:RELATES_TO]->(m:Entity)
2 RETURN n, r, m
3 LIMIT 100
```

也可以直接查看事实文本：

”查看记忆事实文本”

```
1 MATCH ()-[r:RELATES_TO]->()
2 RETURN r.fact, r.valid_at, r.group_id
3 LIMIT 50
```

4.7.3 写入与检索流程

Graphiti 记忆链路如图 4所示。写入阶段，Web 问答在返回答案后由后台线程调用 `add_episode()`，将一轮问答加入 Graphiti；Graphiti 使用 `qwen-plus` 抽取结构化实体和关系，并使用 `DashScope text-embedding-v3` 生成 1024 维向量。检索阶段，系统调用 `search()` 接口，并传入 `group_ids` 和 `num_results=8` 等参数，返回与当前问题最相关的事实边，经过格式化后注入 LLM Prompt。

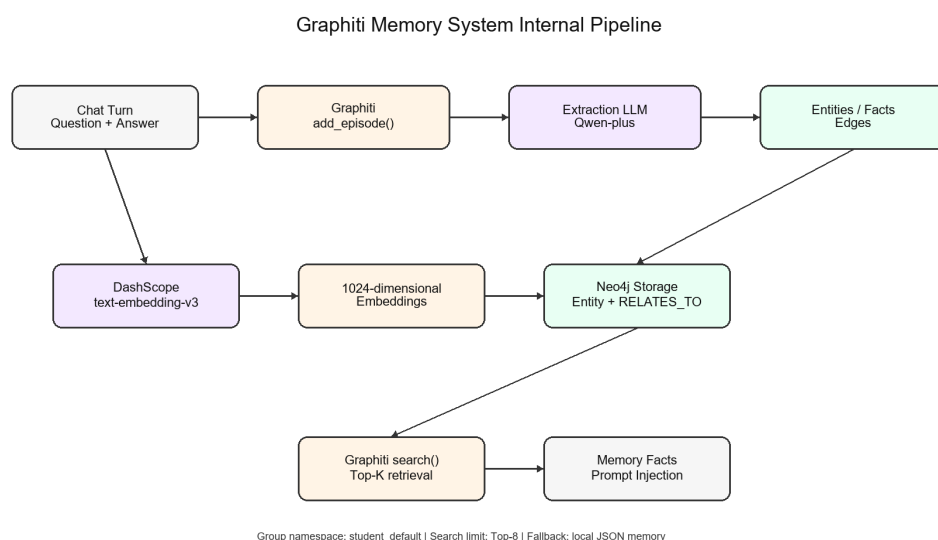


图 4 Graphiti 图记忆写入、抽取、嵌入与检索流程

其核心伪代码如下：

”Graphiti 记忆写入与检索”

```
1 class GraphitiMemoryStore:
2     def format_for_prompt(self, question):
3         facts = self.search(question, top_k=8)
4         return "学生长期记忆(Graphiti):\n" + "\n".join(facts)
5
6     def add_interaction(self, question, answer):
7         episode_body = f"Student: {question}\nAssistant: {answer}"
```

```

8         self.graphiti.add_episode(
9             name="zjut_chat_xxx",
10            episode_body=episode_body,
11            source=EpisodeType.message,
12            source_description="ZJUT learning assistant conversation",
13            reference_time=datetime.now(),
14            group_id="student_default",
15        )
16
17    def search(self, query, top_k=8):
18        return self.graphiti.search(
19            query=query,
20            group_ids=["student_default"],
21            num_results=top_k,
22        )

```

在工程实现中，DashScope text-embedding-v3 接口对单批输入数量有限制，因此适配层将批量文本自动切分为每批不超过 10 条，并加入网络异常重试。若外部 embedding 接口短暂不可用，系统会对失败批次使用确定性 Hash 向量作为兜底，以避免整轮 Graphiti 写入失败。适配层同时对事件循环关闭、Flask 多线程请求和 Graphiti 写入耗时进行防护：主问答请求只等待记忆检索和 LLM 生成，写入阶段在后台完成。该策略保证了课程实验环境下的鲁棒性，同时在网络正常时仍优先使用真实语义向量。

4.7.4 本地部署与 Neo4j 可视化

部署结构如图 5 所示。Graphiti 不是独立服务，而是 Python 依赖库；真正需要单独启动的是 Neo4j 容器。Bot 进程通过 Bolt 协议连接 localhost:7687，用户可以通过 Neo4j Browser 访问 localhost:7474 可视化查看记忆图。

常用启动命令如下：

”Graphiti 记忆系统启动命令”

```

1 docker start zjut-graphiti-neo4j
2 python src/llm_chatbot.py

```

常用 Neo4j 可视化查询如下：

”Neo4j Browser 可视化查询”

```

1 MATCH (n)-[r]->(m)
2 RETURN n, r, m

```

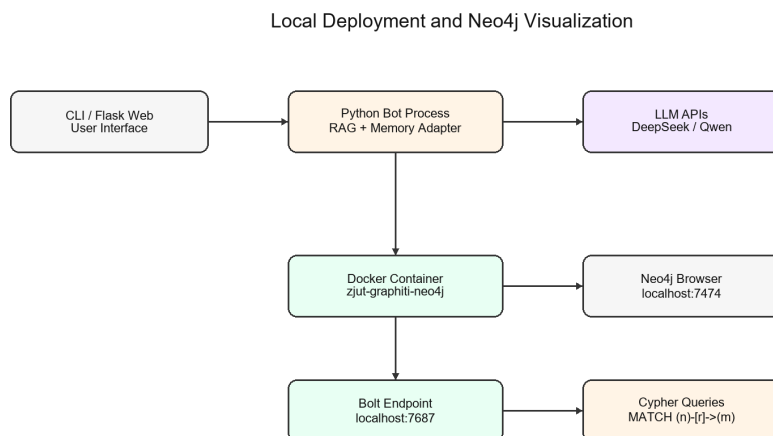


图 5 本地部署与 Neo4j 记忆图可视化关系

3 | LIMIT 100

5 系统展示

5.1 系统架构总览

ZJUT 伴学 Bot 采用前后端分离架构，如图 6所示。系统包括数据层（Neo4j 图数据库与本地学生记忆 JSON）、检索层（向量检索 + 图谱查询）、记忆层（学生画像读取与更新）、生成层（LLM）和交互层（Web/CLI 界面）。

5.2 知识图谱可视化

图 7展示了 Neo4j 中构建的知识图谱结构。图中节点代表不同类型的实体（专业、课程、能力等），边代表实体间的关系（开设、属于、培养等）。可以清晰看到自动化专业与其开设课程、培养能力之间的关联网。

5.3 Web 交互界面

系统提供面向业务应用的 Web 工作台界面，如图 8所示。新版界面采用侧边导航、顶部操作栏、指标概览、快捷问题、主聊天区和系统状态面板的布局，避免早期演示页面中过于突出的上传控件和装饰性视觉元素。用户可以通过自然语言提问，系统先读取 Graphiti 长期记忆和课程知识图谱上下文，再调用主问答 LLM 生成回答。为了降低隐私和误操作风险，Web 端暂不提供成绩单上传或文



图 6 ZJUT 伴学 Bot 系统架构总览

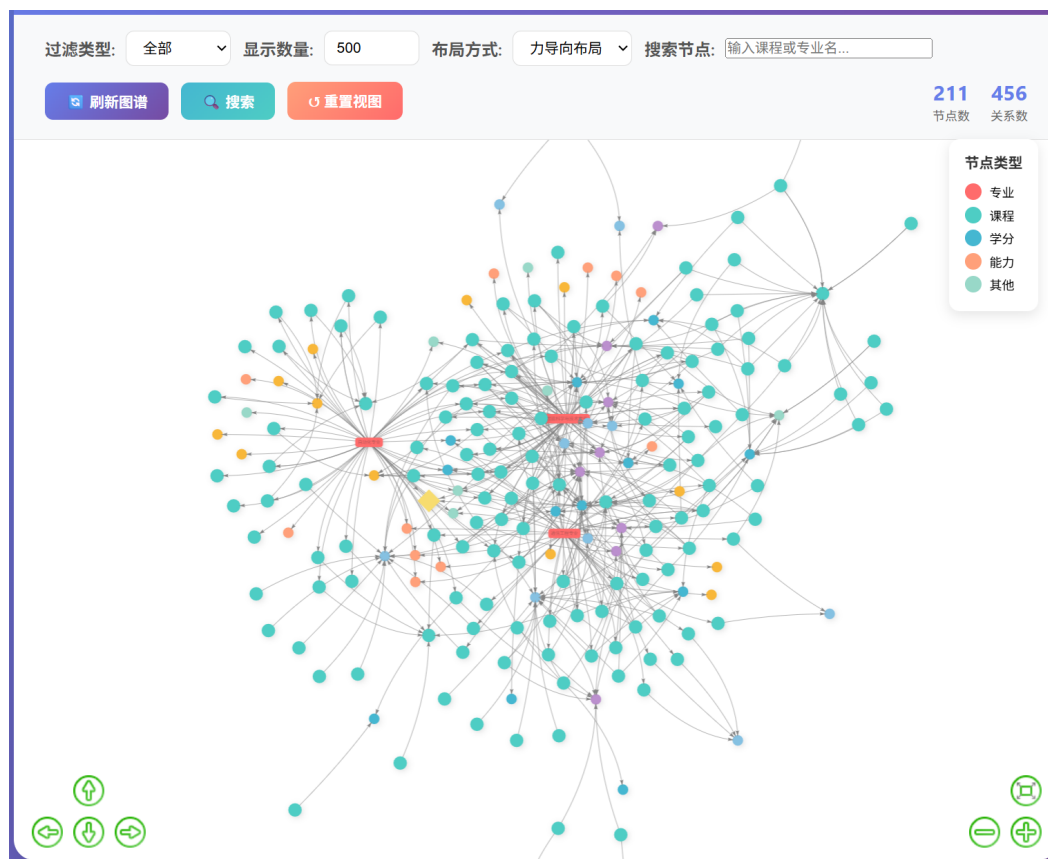


图 7 知识图谱可视化（Neo4j Browser）

本写入长期记忆接口，仅保留“清空对话”和“清空长期记忆”操作；成绩单等个人材料可通过本地命令行脚本或显式对话输入进行维护。

图 9和图 10展示了不同类型问题的问答效果，包括课程查询、前置课程查询、能力培养查询等多种场景。

6 总结与展望

6.1 研究总结

本实验成功构建了“ZJUT 伴学 Bot”——一个面向浙江工业大学信息学院的智能问答机器人，主要成果包括：

- 实现了基于 OneKE 的自动化知识抽取流程，并补充校园与竞赛知识，形成 561 条高质量三元组
- 采用 Neo4j 图数据库构建包含 289 个实体节点的知识图谱，覆盖自动化、智能科学与技术、通信工程三个专业及相关学科竞赛



图 8 Web 问答界面示例

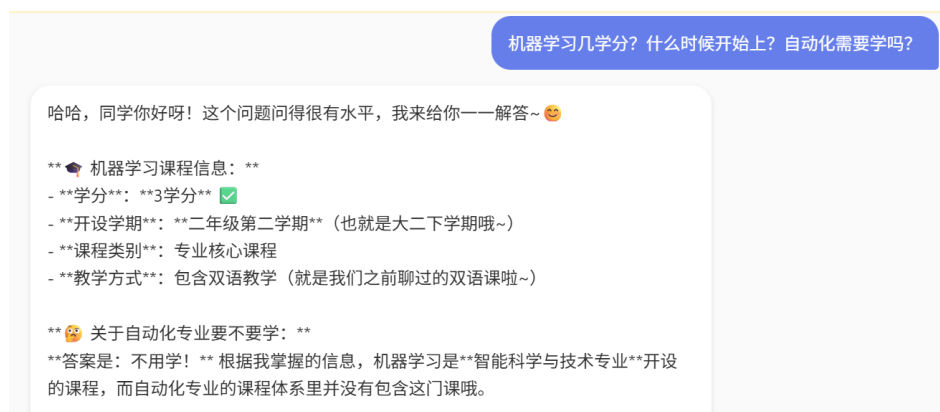


图 9 课程查询问答示例

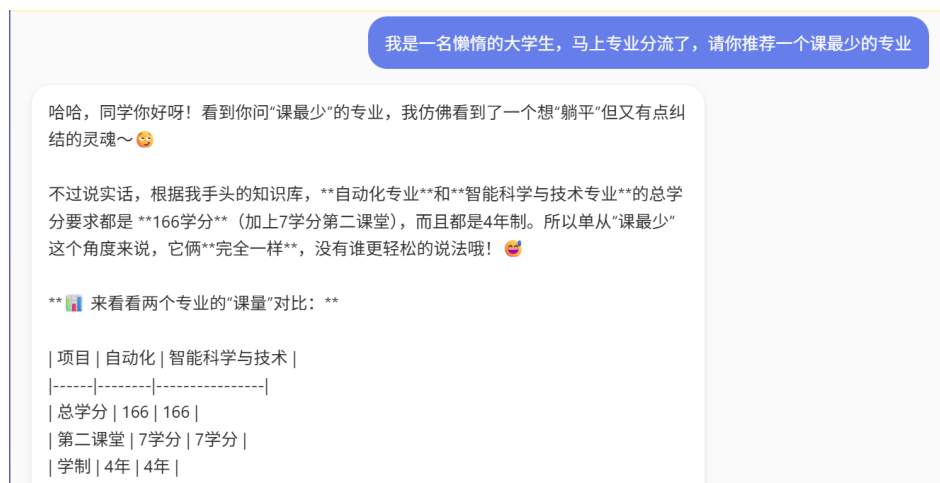


图 10 前置课程查询问答示例

- 设计了向量检索（BGE+Faiss）与图谱查询（Cypher）相结合的混合检索策略
- 基于 RAG 技术实现自然语言问答，当前 Web 主问答模型使用 DeepSeek deepseek-v4-flash，也可切换到 Qwen 或 OpenAI 兼容模型
- 引入本地 JSON 长期记忆模块，并进一步升级为 Graphiti+Neo4j 图记忆，支持学生画像沉淀、事实关系检索和个性化学习建议生成；Web 回答先返回，记忆写入后台异步完成
- 提供业务化 Web 工作台和命令行双界面，支持课程查询、学分要求、前置课程、能力培养等多种问答场景，并保留长期记忆清空能力

6.2 系统特色

ZJUT 伴学 Bot 的核心特色：

- **自动化构建：**OneKE 自动抽取知识，相比人工标注效率提升 10 倍以上
- **图谱可视化：**Neo4j Browser 直观展示专业-课程-能力关系网络
- **智能检索：**混合检索策略结合语义相似度和图结构信息
- **长期记忆：**通过本地 JSON 和 Graphiti 图记忆共同保存学生画像、对话 episode 和实体关系，使机器人能够结合专业、兴趣、已学课程和目标持续优化回答
- **业务化交互：**Web 端采用工作台式布局，保留问答、图谱查看、对话清空和长期记忆清空，避免在公开界面暴露敏感文件上传入口

6.3 改进方向

未来可从以下方向改进：

- **知识扩展：**整合课程大纲、教学计划等多源数据
- **推理增强：**引入图神经网络进行知识推理
- **多模态融合：**在权限控制和隐私保护完善后，再支持课程关系图、培养方案表格和成绩单等材料的受控导入与视觉问答
- **个性化推荐：**进一步结合学生成绩、竞赛经历和长期记忆推荐选课方案
- **增量更新：**设计增量抽取和图谱更新机制

本研究展示了知识图谱与 RAG 技术在教育信息化领域的应用潜力，为构建智能化学业规划系统提供了技术参考。

参考文献

- [1] JI S, PAN S, CAMBRIA E, et al. A survey on knowledge graphs: Representation, acquisition, and applications[J]. IEEE Transactions on Neural Networks and Learning Systems, 2021, 33(2): 494-514.
- [2] YE Y, DU Y, WANG Y, et al. Oneke: A dockerized schema-guided llm agent-based knowledge extraction system[A]. 2024.
- [3] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[C]//Advances in Neural Information Processing Systems: Vol. 33. 2020: 9459-9474.
- [4] ROBINSON I, WEBBER J, EIFREM E. Graph databases: New opportunities for connected data[M]. O'Reilly Media, 2015.
- [5] XIAO S, LIU Z, ZHANG P, et al. C-pack: Packaged resources to advance general chinese embedding[A]. 2023.
- [6] JOHNSON J, DOUZE M, JÉGOU H. Billion-scale similarity search with gpus[J]. IEEE Transactions on Big Data, 2019, 7(3): 535-547.